

Open in app ↗

 Search WriteWelcome Offer Access to everything. Now 30% off. [Upgrade now](#)

Pandas



Python



Data Analysis



Data Science



Machine Learning



Pandas in Python: Your Data's Best Friend



Marishetty Ramyakrishna 16 min read · 1 day ago



From 100,000 Messy Rows to Meaningful Insights — A Beginner-Friendly Journey into Data Analysis

Imagine opening an Excel file with 100,000 rows, hundreds of columns, missing values, duplicate records, and dates written in five different formats.

Would you really want to clean all of that manually?

Absolutely not.

That's where Pandas walks in.

From Messy Data to Meaningful Insights

The Pandas Workflow



What Exactly Is Pandas?

Pandas is an open-source Python library mainly used for **data manipulation and data analysis**.

In simple terms, Pandas helps us work with structured data in an easier and more efficient way.

When we work with real-world datasets, the data may contain thousands or even millions of rows. It can also contain missing values, duplicate records, inconsistent formats, and unnecessary information.

Pandas provides tools to help us handle these problems.

With Pandas, we can:

- **Load datasets from files such as CSV and Excel**
- **Explore and understand datasets**
- **Select specific rows and columns**
- **Filter data based on conditions**
- **Sort data**
- **Handle missing values**
- **Remove duplicate records**
- **Create new columns**
- **Group and aggregate data**
- **Merge multiple datasets**
- **Work with dates**
- **Prepare data for visualization and Machine Learning**

For example, imagine a customer dataset containing:

Name	Age	City	Purchase
Rahul	25	Hyderabad	2500
Priya	30	Chennai	3500
Arjun	28	Mumbai	1800
Sneha	35	Hyderabad	4200

A simple customer dataset represented as a Pandas DataFrame.

Using Pandas, we can answer questions such as:

- What is the average age of customers?
- Which city has the highest purchases?
- Which customers spent more than ₹3,000?
- Are there any missing values?
- Are there duplicate records?
- What is the total purchase amount?

Instead of manually checking every row, Pandas allows us to perform these operations programmatically.

Why Do We Need Pandas?

Imagine you receive a dataset containing **100,000 customer records**.

Your manager asks:

“Find all customers from Hyderabad who spent more than ₹5,000.

Doing this manually would be extremely time-consuming.

With Pandas, we can perform this operation with a simple filtering condition.

The important point is that Pandas doesn't just save time.

It makes our analysis:

- Faster
- Reproducible
- Consistent
- Easier to modify
- Easier to automate

This is why Pandas is widely used in Data Analytics and Data Science.

Installing Pandas

Before using Pandas, we need to install it in our Python environment.

If you are using Command Prompt or Terminal, we can install Pandas using `pip`.

```
pip install pandas
```

Once the installation is complete, Pandas is ready to use.

Importing Pandas

After installing Pandas, we need to import it into our Python program.

Python libraries are imported using the `import` statement.

For Pandas, the commonly used alias is `pd`.

```
import pandas as pd
```

Here:

- `import` tells Python to load the library.
- `pandas` is the library name.
- `pd` is the short alias we use to access Pandas.

Instead of writing:

```
pandas.DataFrame()
```

we can simply write:

```
pd.DataFrame()
```

The `pd` alias is a widely accepted convention in the Python Data Science community.

Pandas Data Structures

Before working with Pandas, we need to understand its two most important data structures:

1. **Series**

2. **DataFrame**

These two structures form the foundation of Pandas.

A simple way to remember them is:

Series = One-dimensional data

DataFrame = Two-dimensional data

What Is a Pandas Series?

A **Series** is a one-dimensional labeled data structure.

You can think of it as a **single column of data**.

For example, suppose we have the marks of four students:

85, 90, 78, and 92.

We can convert these values into a Pandas Series.

```
marks = [85, 90, 78, 92]

series = pd.Series(marks)

print(series)
```

```
0    85
1    90
2    78
3    92
dtype: int64
```

Notice the numbers on the left:

```
0
1
2
3
```

These are called **indexes**.

By default, Pandas starts the index from 0 .

The numbers on the right are the actual values.

Creating a Series with Custom Indexes

We don't always have to use the default numerical indexes.

We can provide our own labels.

For example, suppose the marks belong to different subjects.

```
marks = pd.Series(  
    [85, 90, 78],  
    index=["Python", "SQL", "EDA"]  
)  
  
print(marks)
```

```
Python    85  
SQL       90  
EDA       78  
dtype: int64
```

Now the Series is much easier to understand.

Instead of:

```
0    85  
1    90  
2    78
```

we have:

Python	85
SQL	90
EDA	78

This gives each value a meaningful label.

What Is a DataFrame?

A **DataFrame** is a two-dimensional data structure consisting of rows and columns.

If a Series is like a single column, a DataFrame is like a complete table.

Name	Age	City
Rahul	25	Hyderabad
Priya	30	Chennai
Arjun	28	Mumbai

A simple Pandas DataFrame containing customer names, ages, and cities.

This type of structure can be represented using a Pandas DataFrame.

```
data = {  
    "Name": ["Rahul", "Priya", "Arjun"],  
    "Age": [25, 30, 28],  
    "City": ["Hyderabad", "Chennai", "Mumbai"]  
}  
  
df = pd.DataFrame(data)  
  
print(df)
```

	Name	Age	City
0	Rahul	25	Hyderabad
1	Priya	30	Chennai
2	Arjun	28	Mumbai

Here:

- Name, Age, and City are columns.
- Each person represents a row.
- 0, 1, and 2 are indexes.

This is the structure we will work with most often in real-world Pandas projects.

Feature	85 90 78 Series	DataFrame
Dimension	One-dimensional	Two-dimensional
Structure	Similar to one column	Similar to a complete table
Data	Contains one set of values	Contains multiple columns
Index	Has an index	Has rows, columns, and an index
	Pandas Series 0 85 1 90 2 78 3 92 (1D)	Pandas DataFrame 0 Rahul 25 Hyderabad 1 Priya 30 Chennai 2 Arjun 28 Mumbai 3 Sneha 35 Hyderabad (2D)

A simple comparison of Pandas Series and DataFrame.

Think about an Excel sheet.

A single column is similar to a **Series**.

The entire table is similar to a **DataFrame**.

Reading Real-World Data

Creating DataFrames manually is useful when learning.

But in real Data Analytics projects, we usually don't type the data ourselves.

Instead, we receive datasets from sources such as:

- CSV files
- Excel files
- Databases
- APIs
- JSON files

One of the most common formats is **CSV**.

CSV stands for **Comma-Separated Values**.

Suppose we have a file called:

```
customers.csv
```

Pandas allows us to load this file directly into a **DataFrame**.

```
df = pd.read_csv("customers.csv")
```

Now the data inside the CSV file has been loaded into the **DataFrame** called `df`.

We can then work with that **DataFrame** using **Pandas**.

Looking at the Dataset

After loading a dataset, the first thing we should do is **look at the data**.

We should not immediately start cleaning or modifying it.

First, we need to understand what the dataset contains.

Pandas provides several functions for this purpose.

One of the most commonly used functions is `head()`.

`head()` — Viewing the First Rows

The `head()` function displays the first five rows of a DataFrame by default.

```
df.head()
```

We can also specify how many rows we want to see.

For example, to see the first 10 rows:

```
df.head(10)
```

This is useful because large datasets can contain thousands or millions of rows.

We don't need to display the entire dataset just to understand its structure.

`tail()` — Viewing the Last Rows

The `tail()` function is similar to `head()` , but it displays the last five rows by default.

```
df.tail()
```

We can also specify the number of rows:

```
df.tail(10)
```

This can be useful for checking the beginning and end of a dataset.

shape — Checking Dataset Size

One of the first questions we should ask after loading a dataset is:

How large is my dataset?

For this, we use the `shape` attribute.

```
df.shape
```

Suppose the output is:

```
(1000, 8)
```

This means:

- 1000 rows
- 8 columns

The first value represents rows.

The second value represents columns.

columns — **Checking Column Names**

To see the names of all columns in our DataFrame, we use:

```
df.columns
```

For example, the output might be:

```
Index(['Name', 'Age', 'City', 'Purchase'], dtype='object')
```

This immediately tells us what information is available in the dataset.

info() — Understanding the Dataset

The `info()` function is one of the most important functions during the initial stage of EDA.

It provides information about:

- Number of rows
- Column names
- Non-null values
- Data types
- Memory usage

```
df.info()
```

For example, you might see:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 4 columns):
 #   Column      Non-Null Count  Dtype
---  -
 0   Name        1000 non-null   object
 1   Age         980 non-null    int64
 2   City        995 non-null    object
 3   Purchase    1000 non-null   float64
```

From this information, we can already notice that the `Age` column has only 980 non-null values.

Since the dataset contains 1000 rows:

$1000 - 980 = 20$ missing values

This is one example of how simply understanding the dataset can reveal data-quality problems.

`dtypes` — **Checking Data Types**

We can use `dtypes` to check the data type of every column.

```
df.dtypes
```

Example output:

```
Name      object
Age       int64
City      object
Purchase  float64
dtype: object
```

Some common Pandas data types include:

- `int64` → Integer numbers

- `float64` → Decimal numbers
- `object` → Usually text
- `bool` → True or False
- `datetime` → Date and time

Understanding data types is important because different operations work differently with different types of data.

`describe()` — Statistical Summary

The `describe()` function gives us a statistical summary of numerical columns.

It can provide:

- Count
- Mean
- Standard deviation
- Minimum
- 25th percentile
- Median
- 75th percentile
- Maximum

```
df.describe()
```

For example:

	Age	Purchase
count	980.0000	1000.0000
mean	29.5000	3250.5000
std	5.2000	1450.3000
min	18.0000	500.0000
25%	25.0000	2200.0000
50%	29.0000	3100.0000
75%	34.0000	4200.0000
max	60.0000	9500.0000

This gives us a quick statistical understanding of our numerical data.

Selecting Columns

Once we understand our dataset, we may want to work with specific columns.

To select one column, we use the column name inside square brackets.

```
df["Age"]
```

This returns only the `Age` column.

To select another column:

```
df["City"]
```

Selecting Multiple Columns

We can also select multiple columns at the same time.

```
df[["Name", "Age"]]
```

Here we use **double square brackets** because we are passing a list of column names.

This returns only the `Name` and `Age` columns.

Filtering Data

Selecting columns is useful, but often we need to select **specific rows based on a condition**.

For example:

Find all customers whose age is greater than 25.

Pandas allows us to do this using Boolean filtering.

```
df[df["Age"] > 25]
```

Pandas checks the condition for every row.

If the condition is `True`, that row is returned.

If the condition is `False`, that row is excluded.

Filtering with Multiple Conditions

We can also combine multiple conditions.

For **AND**, we use:

&

For **OR**, we use:

|

For example, suppose we want customers who are:

- Older than 25
- AND from Hyderabad

```
df[(df["Age"] > 25) &
```

```
(df["City"] == "Hyderabad")  
]
```

For an OR condition:

```
df[  
    (df["City"] == "Hyderabad") |  
    (df["City"] == "Mumbai")  
]
```

This returns customers from either Hyderabad or Mumbai.

Sorting Data

After filtering data, another common task is **sorting**.

Sorting helps us arrange our data in a meaningful order.

For example, we may want to:

- Arrange customers from youngest to oldest
- Find customers with the highest purchases
- Arrange products from lowest price to highest price
- Find the top-performing records

Pandas provides the `sort_values()` function for this purpose.

Sorting by Age

To sort the DataFrame based on the `Age` column:

```
df.sort_values("Age")
```

By default, Pandas sorts the values in **ascending order**.

That means the smallest value appears first.

Sorting in Descending Order

Sometimes we want the largest value first.

For that, we use:

```
df.sort_values("Age", ascending=False)
```

Here, `ascending=False` tells Pandas to sort the values from largest to smallest.

Sorting by Purchase Amount

We can also sort customers based on their purchase amount.

```
df.sort_values("Purchase", ascending=False)
```

This will show the customers with the highest purchase amounts at the top.

Sorting is especially useful when we want to identify **top customers, highest sales, lowest prices, or other extreme values**.

Cleaning Messy Data

Real-world data is rarely perfect.

A dataset may contain:

- Missing values
- Duplicate records
- Incorrect data types
- Inconsistent text
- Unnecessary columns
- Incorrect formats
- Unexpected values

This process of identifying and fixing these problems is called **data cleaning**.

Data cleaning is an important part of Data Analysis because the quality of our analysis depends heavily on the quality of our data.

A simple rule to remember is:

Clean data → Better analysis → Better insights

Pandas provides many functions that make data cleaning easier.

Finding Missing Values

Missing values are very common in real-world datasets.

For example, a customer dataset :

Here, Priya's age is missing.

Before deciding what to do with missing values, we first need to find them.

Pandas provides the `isnull()` function to identify missing values.

```
df.isnull()
```

This returns `True` where a value is missing and `False` where a value is present.

However, for a large dataset, looking at every `True` and `False` value is not practical.



Instead, we can count the missing values in each column.

```
df.isnull().sum()
```

Example Output

```
Name      0
Age       20
City       5
Purchase   0
dtype: int64
```

This tells us that:

- Name has 0 missing values
- Age has 20 missing values
- City has 5 missing values
- Purchase has 0 missing values

Now we know exactly where the missing data exists.

Removing Missing Values

Sometimes missing records are not useful for our analysis.

In such cases, we can remove rows containing missing values using `dropna()`.

```
df.dropna()
```

This returns a DataFrame with rows containing missing values removed.

However, we should not blindly remove missing values.

If a dataset contains thousands of rows and only a few missing records, removing them may be reasonable.

But if a large percentage of the data is missing, removing those rows could result in significant information loss.

Therefore, we should understand the data before deciding how to handle missing values.

Filling Missing Values

Instead of removing missing values, we can sometimes replace them with meaningful values.

This process is called **imputation**.

For numerical columns, one common approach is to use the **mean**.

For example, if some customer ages are missing, we can replace them with the average age.

```
df["Age"] = df["Age"].fillna(df["Age"].mean())
```

Here:

- `df["Age"]` selects the Age column
- `mean()` calculates the average age
- `fillna()` replaces missing values
- The result is assigned back to the Age column

For categorical columns such as City, we can use the most frequently occurring value, called the **mode**.

```
df["City"] = df["City"].fillna(df["City"].mode()[0])
```

Here, `mode()[0]` selects the most frequent value.

The important lesson is that **there is no single method that should be used for every missing value**.

The correct approach depends on the dataset and the meaning of the column.

Finding Duplicate Records

Another common data-quality problem is **duplicate records**.

For example, the same customer may accidentally appear twice in a dataset.

Pandas provides the `df.duplicated()` function to identify duplicate rows.

```
df.duplicated()
```

This returns `True` for duplicate rows.

If we want to count the total number of duplicate rows:

```
df.duplicated().sum()
```

Example Output

```
12
```

This means the DataFrame contains 12 duplicate rows.

Removing Duplicate Records

Once we identify duplicate records, we can remove them using

```
drop_duplicates() .
```

```
df = df.drop_duplicates()
```

This creates a DataFrame without duplicate rows.

Removing duplicates can prevent the same record from being counted multiple times during analysis.

However, we should always check whether repeated records are actually duplicates before deleting them.

Creating a New Column

One of the most useful features of Pandas is the ability to create new columns.

Suppose our dataset contains a `Purchase` column.

We want to calculate a new price after giving customers a 10% discount.

We can create a new column called `Discounted_Price`.

```
df["Discounted_Price"] = df["Purchase"] * 0.90
```

Here, we multiply the purchase amount by `0.90` because the customer pays **90% of the original amount after a 10% discount**.

For example:

```
Purchase = 1000
```

```
Discounted Price = 1000 × 0.90  
                  = 900
```

Pandas performs this calculation for every row automatically.

This is one of the reasons Pandas is so useful for data transformation.

Renaming Columns

Sometimes column names are unclear or difficult to understand.

For example, we may have a column called:

```
Purchase
```

We may want to rename it to:

```
Purchase_Amount
```

Pandas provides the `rename()` function for this.

```
df.rename(columns={"Purchase": "Purchase_Amount"}, inplace=True)
```

Here:

- `Purchase` is the existing column name
- `Purchase_Amount` is the new name
- `inplace=True` applies the change directly to the DataFrame

Clear column names make a dataset easier to understand and work with.

Removing Unwanted Columns

Sometimes a dataset contains columns that are not required for our analysis.

For example, suppose the `city` column is not needed.

We can remove it using the `drop()` function.

```
df.drop("City", axis=1, inplace=True)
```

Here:

- `"City"` is the column we want to remove
- `axis=1` tells Pandas that we are working with a column
- `inplace=True` applies the change directly

Remember:

axis=0 → **rows**

axis=1 → **columns**

Removing unnecessary columns can make a dataset simpler and easier to work with.

GroupBy — Turning Data into Insights

So far, we have worked with individual rows and columns.

But Data Analysis often requires us to answer questions such as:

- How much did each city purchase?
- What is the average purchase amount by city?
- Which city has the highest sales?
- How many customers are from each city?

This is where **GroupBy** becomes extremely useful.

The `groupby()` function allows us to divide data into groups and perform calculations on those groups.

Think of it like this:

Group → Calculate → Compare

Total Purchase by City

Suppose we want to calculate the total purchase amount for each city.

```
df.groupby("City")["Purchase"].sum()
```

Example Output

```
City
Chennai      85000
Hyderabad    125000
Mumbai       95000
Name: Purchase, dtype: int64
```

Now we can easily compare the total purchases between cities.

Average Purchase by City

We can also calculate the average purchase amount for each city.

```
df.groupby("City")["Purchase"].mean()
```

This tells us the average purchase value for each city.

Maximum Purchase by City

To find the highest purchase amount in each city:

```
df.groupby("City")["Purchase"].max()
```

This can help identify the highest-value customer transaction in each city.

Number of Customers by City

We can also count records in each group.

```
df.groupby("City")["Name"].count()
```

This tells us how many customer records belong to each city.

GroupBy is one of the most important concepts in Pandas because it allows us to turn raw data into meaningful summaries.

Combining Data with `merge()`

In real-world projects, information is often stored in multiple datasets.

For example, we might have one DataFrame containing customer information:

Customer Information	
Customer_ID	Name
101	Rahul
102	Priya
103	Arjun

And another DataFrame containing purchase information:

Customer_ID	Purchase
101	2500
102	3500
103	1800

Both datasets contain `Customer_ID`.

We can use this common column to combine the datasets.

Pandas provides the `merge()` function for this purpose.

```
result = pd.merge(  
    customers,  
    purchases,  
    on="Customer_ID"  
)
```

Here:

- `customers` is the first DataFrame
- `purchases` is the second DataFrame
- `Customer_ID` is the common column
- `result` stores the combined DataFrame

This is similar to joining tables in SQL.

Combining Data with `concat()`

Sometimes we have datasets with the same columns and simply want to put them together.

For example, suppose we have January and February sales data.

We can combine them using `concat()`.

```
df_final = pd.concat([df_january, df_february])
```

This combines the rows from both DataFrames.

`concat()` is useful when datasets have a similar structure and need to be stacked together.

Working with Dates

Dates are extremely common in real-world datasets.

For example:

```
2026-01-15  
2026-02-20  
2026-03-10
```

Sometimes dates are stored as ordinary text.

Before performing date-based analysis, it is useful to convert them into Pandas' datetime format.

```
df["Date"] = pd.to_datetime(df["Date"])
```

Now Pandas can understand the column as dates.

Once the column has been converted, we can extract useful information from it.

Extracting the Year

We can extract the year from a date using `.dt.year`.

```
df["Year"] = df["Date"].dt.year
```

This creates a new column containing the year.

Extracting the Month

We can extract the month using `.dt.month`.

```
df["Month"] = df["Date"].dt.month
```

This gives us the month number.

Extracting the Day

We can extract the day using `.dt.day`.

```
df["Day"] = df["Date"].dt.day
```

This gives us the day of the month.

Extracting the Day Name

We can also find the name of the day.

```
df["Day_Name"] = df["Date"].dt.day_name()
```

For example:

Monday
Tuesday
Wednesday

Working with dates becomes especially useful when analyzing **monthly sales, yearly trends, daily transactions, or seasonal patterns.**

Pandas and Exploratory Data Analysis

Pandas is one of the most important tools used during **Exploratory Data Analysis**, commonly called EDA.

EDA is the process of exploring a dataset to understand:

- Its structure
- Data types
- Missing values
- Duplicate records
- Statistical properties
- Patterns
- Relationships
- Possible data-quality problems

A typical first-level EDA workflow might look like this:

Load → Understand → Clean → Explore → Analyze → Visualize

Let's combine some of the Pandas functions we learned.

```
import pandas as pd
```

```
df = pd.read_csv("customers.csv")
```

First, we load the dataset.

```
df.shape
```

This tells us the number of rows and columns.

```
df.columns
```

This shows the column names.

```
df.info()
```

This helps us understand data types and missing values.

```
df.head()
```

This shows the first few records.

```
df.describe()
```

This gives us a statistical summary of numerical columns.

```
df.isnull().sum()
```

This helps us identify missing values.

```
df.duplicated().sum()
```

This tells us how many duplicate records exist.

These simple commands provide a strong starting point for understanding a dataset before performing deeper analysis.

Pandas and the Data Science Ecosystem

Pandas is powerful, but Data Science does not rely on Pandas alone.

Different Python libraries are commonly used for different tasks.

Pandas

Used mainly for:

- Data manipulation
- Data cleaning
- Data analysis
- Working with structured datasets

NumPy

Used mainly for:

- Numerical computing
- Arrays
- Mathematical operations

Matplotlib

Used for:

- Data visualization
- Charts
- Graphs

Seaborn

Used for:

- Statistical visualization
- Attractive and informative charts
- Exploring relationships between variables

Scikit-learn

Used for:

- Machine Learning
- Data preprocessing
- Model training
- Model evaluation

A simple way to think about the ecosystem is:

Pandas → Work with data

NumPy → Perform numerical operations

Matplotlib / Seaborn → Visualize data

Scikit-learn → Build Machine Learning models

These libraries often work together in a Data Science project.

Saving Your Data

After cleaning or transforming a dataset, we may want to save the result.

Pandas allows us to save a DataFrame in different formats.

One of the most common formats is CSV.

Saving as CSV

```
df.to_csv("cleaned_data.csv", index=False)
```

The `index=False` prevents Pandas from saving the DataFrame index as an additional column.

We can also save a DataFrame as an Excel file.

Saving as Excel

```
df.to_excel("cleaned_data.xlsx", index=False)
```

This is useful when the cleaned dataset needs to be shared with others or used in another application.

Pandas Cheat Sheet

After learning the fundamentals, here is a quick reference for some commonly used Pandas functions.

Pandas Cheat Sheet

Pandas provides data analysis tools for Python. All of the following code examples refer to the dataframe below.

df =

	col1	col2
A	1	4
B	2	5
C	3	6

← axis 1
← axis 0

Getting Started

Import pandas:

```
import pandas as pd
```

Create a series:

```
s = pd.Series([1, 2, 3],
              index=['A', 'B', 'C'],
              name='col1')
```

Create a dataframe:

```
data = [[1, 4], [2, 5], [3, 6]]
index = ['A', 'B', 'C']
df = pd.DataFrame(data, index=index,
                  columns=['col1', 'col2'])
```

Read a csv file with pandas:

```
df = pd.read_csv('filename.csv')
```

Advanced parameters:

```
df = pd.read_csv('filename.csv', sep=',',
                 names=['col1', 'col2'],
                 index_col=0,
                 encoding='utf-8',
                 nrows=3)
```

Selecting rows and columns

Select single column:

```
df['col1']
```

Select multiple columns:

```
df[['col1', 'col2']]
```

Show first n rows:

```
df.head(2)
```

Show last n rows:

```
df.tail(2)
```

Select rows by index values:

```
df.loc['A'] df.loc[['A', 'B']]
```

Select rows by position:

```
df.loc[1] df.loc[1:]
```

Data wrangling

Filter by value:

```
df[df['col1'] > 1]
```

Sort by one column:

```
df.sort_values('col1')
```

Sort by columns:

```
df.sort_values(['col1', 'col2'],
               ascending=[False, True])
```

Identify duplicate rows:

```
df.duplicated()
```

Identify unique rows:

```
df['col1'].unique()
```

Swap rows and columns:

```
df = df.transpose()
df = df.T
```

Drop a column:

```
df = df.drop('col1', axis=1)
```

Clone a data frame:

```
clone = df.copy()
```

Connect multiple data frames vertically:

```
df2 = df + 5 #new dataframe
pd.concat([df, df2])
```

Merge multiple data frames horizontally:

```
df3 = pd.DataFrame([[1, 7], [8, 9]],
                  index=['B', 'D'],
                  columns=['col1', 'col3'])
```

#df3: new dataframe

Only merge complete rows (INNER JOIN):

```
df.merge(df3)
```

Left column stays complete (LEFT OUTER JOIN):

```
df.merge(df3, how='left')
```

Right column stays complete (RIGHT OUTER JOIN):

```
df.merge(df3, how='right')
```

Preserve all values (OUTER JOIN):

```
df.merge(df3, how='outer')
```

Merge rows by index:

```
df.merge(df3, left_index=True,
        right_index=True)
```

Fill NaN values:

```
df.fillna(0)
```

Apply your own function:

```
def func(x):
    return 2**x
df.apply(func)
```

Arithmetics and statistics

Add to all values:

```
df + 10
```

Sum over columns:

```
df.sum()
```

Cumulative sum over columns:

```
df.cumsum()
```

Mean over columns:

```
df.mean()
```

Standard deviation over columns:

```
df.std()
```

Count unique values:

```
df['col1'].value_counts()
```

Summarize descriptive statistics:

```
df.describe()
```

A quick Pandas cheat sheet featuring commonly used functions

This cheat sheet is not meant to replace understanding.

Instead, use it as a quick reference while working on projects.

The Most Important Thing

Learning Pandas is not about memorizing hundreds of functions.

The real goal is to understand what problem you are trying to solve.

For example:

I want to see my data →

```
df.head()
```

I want to know the dataset size →

```
df.shape
```

I want to find missing values →

```
df.isnull().sum()
```

I want to remove duplicates →

```
df.drop_duplicates()
```

I want to filter customers →

```
df[df["Age"] > 25]
```

I want to group data →

```
df.groupby("City")["Purchase"].sum()
```

Once you start thinking in terms of **questions and problems**, Pandas becomes much easier to learn.

Conclusion:

Pandas is one of the most useful Python libraries for anyone working with data.

From loading a simple CSV file to cleaning thousands of messy records, Pandas gives us the tools needed to work with data efficiently.

We started with simple concepts such as **Series and DataFrames** and gradually moved toward:

- Reading datasets
- Exploring data
- Selecting columns
- Filtering rows
- Sorting data
- Cleaning missing values
- Removing duplicates
- Creating new columns
- Grouping data

- Combining datasets
- Working with dates
- Performing basic EDA
- Saving cleaned data

But this is only the beginning.

The real power of Pandas comes when we start using these concepts together on **real-world datasets**.

Instead of looking at a dataset as thousands of confusing rows, we can start asking questions:

What does the data tell us?

What patterns can we find?

What problems exist in the data?

What insights can we extract?

That is where Pandas becomes more than just a Python library.

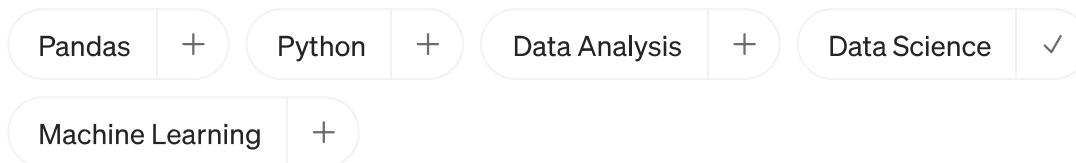
It becomes a powerful tool for turning **raw data into meaningful information and insights**.

Keep Learning

This article is only the beginning of the journey. The next step is to take these Pandas concepts and apply them to **real-world datasets and Exploratory Data Analysis (EDA)**.

Happy Learning!

If this article helped you understand Pandas, feel free to **clap** , **share it**, and **follow me** for more content on Python, Data Analysis, Machine Learning, and Data Science.



Written by Marishetty Ramyakrishna

0 followers · 2 following

Follow

